```
            if stale_seconds is not None and (now - last_seen) > stale_seconds:
                stale_keys.append(key)
                continue

            clean = {k: v for k, v in value.items() if not k.startswith("_")}
            rows.append(clean)

        for key in stale_keys:
            self._state.pop(key, None)

        return rows


class PingstationStreamReceiver:
    """
    TCP listener that receives live Pingstation bytes and keeps a decoded snapshot.
    The PingStation device must be configured to connect TO this PC on the
    configured port (default 30003).  The PC acts as the TCP server.
    """

    def __init__(self, host, port, decoder=None, stale_seconds=45, raw_logger=None):
        self.host = host
        self.port = port
        self.stale_seconds = stale_seconds
        self.decoder = decoder or PingstationRawDecoder()
        self.raw_logger = raw_logger

        self._lock = threading.Lock()
        self._pending_records = []
        self._stop_event = threading.Event()
        self._thread = None
        self._server_socket = None
        self._client_socket = None
        self._socket_lock = threading.Lock()

        # ---- connection-status tracking (exposed via get_connection_status) ----
        self._status_lock = threading.Lock()
        self._is_connected = False          # True while a client socket is open
        self._last_data_time = None         # epoch float, last time bytes arrived
        self._total_frames_decoded = 0
        self._total_connections = 0
        self._bind_error = None             # last OSError message from bind/listen

    # ------------------------------------------------------------------
    def get_connection_status(self):
        """Return a dict describing the live receiver state."""
        with self._status_lock:
            return {
                "connected": self._is_connected,
                "lastDataTime": self._last_data_time,
                "totalFramesDecoded": self._total_frames_decoded,
                "totalConnections": self._total_connections,
                "bindError": self._bind_error,
                "listenPort": self.port,
                "listenHost": self.host,
            }

    # ------------------------------------------------------------------
    def start(self):
        if self._thread and self._thread.is_alive():
            return
        self._thread = threading.Thread(
            target=self._run, name="pingstation-listener", daemon=True
        )
        self._thread.start()

    def stop(self):
        self._stop_event.set()
        with self._socket_lock:
            sockets = (self._client_socket, self._server_socket)
        for sock in sockets:
            if sock is None:
                continue
```